

Assignment & Submission Workflow

1. Feature Name

Assignment & Submission Workflow

2. Purpose

- **Problem Solved:** Classrooms need structured coursework: instructors publish assignments with due dates, students upload work files, and instructors review, grade, return, or approve that work through a controlled status workflow.
- **Why It Exists:** This feature gives the platform a complete assessment loop beyond quizzes. It enforces file constraints, tracks late submissions, and gates the visibility of grades/feedback until the instructor publishes the evaluation.
- **Role in Application:** It operates inside the Classroom Engine's scope. Assignments belong to classrooms; submissions belong to assignments and users. The AI feedback feature (Feature 9) builds on top of submissions.

3. What the User Can Do

Instructor

- Create / edit / delete assignments in a classroom (title, description, due date).
- View all students' submissions for an assignment (one row per student, including non-submitters).
- Preview and download submission files.
- Verify a submission (move to `UNDER_REVIEW`).
- Mark reviewed / transition status.
- Grade a submission (grade + feedback) → `GRADED`.
- Return a submission for resubmission with feedback → `RETURNED_FOR_RESUBMISSION`.
- Complete a submission (auto-decide Graded vs Returned based on grade threshold).
- Publish the evaluation (make grade/feedback visible to the student).

Student

- Upload a submission file (`.pdf`, `.doc`, `.docx`, `.txt`, ≤ 5 MB).
- Replace a submitted file (subject to status rules).
- Cancel a submission that is still `SUBMITTED` / `SUBMITTED_AGAIN`.
- View their own submission and, once published, its grade, feedback, and AI feedback.

4. Feature Workflow

```
[Instructor creates assignment]
→ POST /classrooms/{id}/assignments (requires platform Admin role via EnsureAdminAsync)
→ Assignment row created with title, description, due date

[Student uploads]
→ POST /classrooms/{cid}/assignments/{aid}/submit (multipart file)
→ FileValidationService.Validate: size ≤ 5MB, extension in {pdf,doc,docx,txt},
  MIME allowed, MIME matches extension
→ FileStorageService.StoreAsync: writes file to uploads/assignments/assignment-{aid}/student-{uid}/submission.{ext}
  (path-traversal guarded via GetFullPath.StartsWith(_uploadRoot))
→ IsLate computed: DueDate <.UtcNow
→ Submission row created/updated (Status = SUBMITTED, or SUBMITTED_AGAIN if returning)
→ SUBMISSION_UPLOADED audit

[Instructor reviews & grades]
→ PUT .../submissions/{sid}/verify → SUBMITTED → UNDER_REVIEW
→ PUT .../submissions/{sid}/grade {grade, feedback} → GRADED
→ PUT .../submissions/{sid}/complete → GRADED (grade ≥ 5) else RETURNED_FOR_RESUBMISSION
→ PUT .../submissions/{sid}/verify is separate from transition
→ POST .../submissions/{sid}/publish → PublishedAt = now (evaluation visible to student)
```

```
[Student views result]
→ GET /classrooms/{cid}/assignments/{aid}/submission → grade/feedback included only when PublishedAt != null
```

5. How It Works Internally

Assignment CRUD

- `CreateAssignmentAsync` requires platform `Admin` role (`EnsureAdminAsync`) and membership.
- `UpdateAssignmentAsync` / `DeleteAssignmentAsync` require the per-classroom `Instructor` role (`EnsureInstructorAsync`).

File Validation

`FileValidationService.Validate(IFormFile)` checks in order:

1. **Size** $\leq$ `UploadSettings.MaxFileSize` (default 5 MB).
2. **Non-empty**.
3. **Extension** must be in `AllowedExtensions` = `.pdf`, `.doc`, `.docx`, `.txt`.
4. **MIME type** must be in `AllowedMimeTypes`.
5. **Extension/MIME match** — the actual `ContentType` must equal the expected MIME for the extension (e.g. `.pdf` $\Rightarrow$ `application/pdf`). This rejects disguised files.

File Storage (with path-traversal protection)

`FileStorageService.StoreAsync`:

- Relative path = `assignments/assignment-{assignmentId}/student-{userId}`.
- Computes `fullDir` = `Path.GetFullPath(Combine(_uploadRoot, relativePath))` and rejects if it does not start with `_uploadRoot` (path-traversal guard).
- Writes to a `.tmp` file first, then atomically replaces the final file (`submission.{ext}`) — old file deleted before move.
- On failure the temp file is removed.
- `GetStreamAsync` / `DeleteAsync` re-apply the same `StartsWith(_uploadRoot)` guard.

Late Detection

On upload, `isLate` = `assignment.DueDate < DateTime.UtcNow`.

Submission Status Lifecycle

`SubmissionStatus` constants: `NOT_SUBMITTED`, `SUBMITTED`, `UNDER_REVIEW`, `REVIEWED`, `GRADED`, `RETURNED_FOR_RESUBMISSION`, `SUBMITTED_AGAIN`.

- Upload with no existing submission $\rightarrow$ `SUBMITTED`.
- Re-upload after `RETURNED_FOR_RESUBMISSION` $\rightarrow$ `SUBMITTED_AGAIN`; otherwise re-upload of a `SUBMITTED` / `SUBMITTED_AGAIN` $\rightarrow$ `SUBMITTED` (old file deleted if the path changed).
- Replacing a submission in `UNDER_REVIEW` / `REVIEWED` / `GRADED` is blocked (`"Cannot replace submission in its current status."`).
- `VerifySubmissionAsync` $\rightarrow$ `UNDER_REVIEW`.
- `TransitionStatusAsync` allows only `UNDER_REVIEW` and `REVIEWED` targets with strict source checks:
 - $\rightarrow$ `UNDER_REVIEW` only from `SUBMITTED` / `SUBMITTED_AGAIN`.
 - $\rightarrow$ `REVIEWED` only from `UNDER_REVIEW`.
- `GradeSubmissionAsync` sets `Grade`, `Feedback`, `Status` = `GRADED`.
- `ReturnForResubmissionAsync` $\rightarrow$ `RETURNED_FOR_RESUBMISSION` (blocked on a `GRADED` submission or one with no file).
- `CompleteSubmissionAsync` requires a grade set; then `Status` = `Grade >= 5 ? GRADED : RETURNED_FOR_RESUBMISSION`.
- `PublishEvaluationAsync` requires `GRADED`, sets `PublishedAt` = `UtcNow`.

Visibility of Evaluation (publishing gate)

In `GetMySubmissionAsync`:

- `Feedback` and `Grade` are returned to the student **only if** `PublishedAt` is not null.
- AI feedback (Feature 9) is also only attached when published.
- Instructors see grades/feedback regardless (their own list endpoint).

Submission Listing (per classroom)

`GetSubmissionsAsync` lists all `Student`-role classroom members joined to their submissions (or `NOT_SUBMITTED`), ordered by `SubmittedAt` descending. This gives instructors a roster-style view.

Cancellation

`DeleteSubmissionAsync` — only allowed when status is `SUBMITTED` or `SUBMITTED_AGAIN`; deletes the row (the file is not removed from disk by this method).

File Preview / Download Authorization

`GetAuthorizedFileStreamAsync`:

- Loads the submission with assignment/classroom.
- Authorizes if the caller is the submission owner **or** an `Instructor` of the classroom.
- Returns stream + original file name + MIME. Preview sets `X-Content-Type-Options: nosniff`.

6. Frontend Implementation

Layer	File Path	Responsibility
Page	<code>frontend/src/pages/classroom/AssignmentDetailPage.tsx</code>	Assignment detail: upload card, status chips, grade/publish actions
Component	<code>frontend/src/components/classroom/AssignmentCard.tsx</code>	Assignment card in classroom detail
Component	<code>frontend/src/components/classroom/AssignmentUploadCard.tsx</code>	File upload UI
Component	<code>frontend/src/components/classroom/AiFeedbackSection.tsx</code>	AI feedback panel (published evaluations)
Service	<code>frontend/src/services/classroomService.ts</code>	Assignment/submission/grade/publish/preview/download/return calls
Types	<code>frontend/src/types/classroom.types.ts</code>	Assignment, Submission, AiFeedbackResponse types

7. Backend Implementation

Layer	File Path	Responsibility
Controller	<code>backend/Controllers/ClassroomController.cs</code>	Assignments CRUD, submit, grade, verify, complete, publish endpoints
Controller	<code>backend/Controllers/SubmissionController.cs</code>	Preview/download file, transition status, return, AI feedback endpoints
Service	<code>backend/Services/Classroom/ClassroomService.cs</code>	Assignment + submission business logic, status transitions, authorization
Service	<code>backend/Services/Submission/FileValidationService.cs</code>	File size/extension/MIME validation
Service	<code>backend/Services/Submission/FileStorageService.cs</code>	Secure local filesystem storage with path-traversal guard
Config	<code>backend/Configuration/UploadSettings.cs</code>	Upload directory, max size, allowed extensions/MIME types
DTO	<code>backend/DTOs/Classroom/AssignmentDto.cs</code>	Assignment/submission DTOs and transition DTOs

8. Database Implementation

Entity/Table	Purpose	Important Relationship
Assignments	Assignment metadata (title, description, due date)	FK ClassroomId → Classrooms; 1→N Submissions
Submissions	Student file submission + status + evaluation	FK AssignmentId → Assignments; FK UserId → Users; Status, IsLate, Grade, Feedback, PublishedAt, StoredFilePath

- Files are stored on the local filesystem under {UploadDirectory}/assignments/assignment-{id}/student-{userId}/submission.{ext}; the DB stores the relative path.
- Submission.Status is a string (constants in SubmissionStatus), not a database enum.

9. Security & Authorization

- All endpoints require authentication.
- Assignment create requires the platform Admin role; assignment update/delete and submission grading/verification/completion/publishing/return require the per-classroom Instructor role.
- Upload/own-submission view/cancel are scoped to the authenticated user.
- File preview/download requires the submission owner or an instructor of the classroom.
- File validation blocks wrong types/sizes and MIME/extension mismatches; storage path-traversal is prevented at the filesystem layer.
- Grade/feedback are hidden from students until the instructor publishes the evaluation.

10. Important Business Rules

- Allowed file types:** .pdf, .doc, .docx, .txt; max 5 MB; MIME must match extension.
- Late flag:** submission is late when DueDate < now (no blocking — late is allowed and recorded).
- Replace restriction:** cannot replace a submission once it is UNDER_REVIEW, REVIEWED, or GRADED.
- Cancel restriction:** only SUBMITTED / SUBMITTED_AGAIN submissions can be cancelled.
- Transition matrix:** SUBMITTED/SUBMITTED_AGAIN → UNDER_REVIEW → REVIEWED; grade/complete produce GRADED or RETURNED_FOR_RESUBMISSION.
- Completion threshold:** grade ≥ 5 → GRADED; grade < 5 → RETURNED_FOR_RESUBMISSION (requires grade set first).
- Publishing gate:** evaluation (grade/feedback/AI feedback) visible to students only after publish.
- Return blocked** for GRADED submissions or submissions without an uploaded file.

11. Example of Internal Execution

Step 1: Student uploads a PDF

- User Action:** Student selects essay.pdf (due date already passed) and submits.
- Frontend:** classroomService.uploadSubmission → POST /classrooms/4/assignments/10/submit.
- Validation:** size ok, .pdf allowed, ContentType = application/pdf matches .pdf.
- Storage:** file written to uploads/assignments/assignment-10/student-u123/submission.pdf.
- DB:** Submission row created: Status = SUBMITTED, IsLate = true, SubmittedAt = now.

Step 2: Instructor verifies and grades

- PUT /classrooms/4/assignments/10/submissions/88/verify → status UNDER_REVIEW.
- PUT /classrooms/4/assignments/10/submissions/88/grade { grade: 8, feedback: "Good work" } → status GRADED.
- POST /classrooms/4/assignments/10/submissions/88/publish → PublishedAt set.

Step 3: Student views the evaluation

- GET /classrooms/4/assignments/10/submission → because PublishedAt != null, Grade = 8, Feedback = "Good work", and AI feedback (if generated) are included in the response.

Step 4: Low grade auto-return

- A submission graded 3 via complete becomes RETURNED_FOR_RESUBMISSION; the student re-uploads → status SUBMITTED_AGAIN.

12. Files Involved

Frontend

- `frontend/src/pages/classroom/AssignmentDetailPage.tsx`
- `frontend/src/components/classroom/AssignmentCard.tsx`
- `frontend/src/components/classroom/AssignmentUploadCard.tsx`
- `frontend/src/components/classroom/AiFeedbackSection.tsx`
- `frontend/src/services/classroomService.ts`
- `frontend/src/types/classroom.types.ts`

Backend

- `backend/Controllers/ClassroomController.cs`
- `backend/Controllers/SubmissionController.cs`
- `backend/Services/Classroom/ClassroomService.cs`
- `backend/Services/Submission/FileValidationService.cs`
- `backend/Services/Submission/FileStorageService.cs`
- `backend/Configuration/UploadSettings.cs`
- `backend/DTOs/Classroom/AssignmentDto.cs`

Database

- `backend/Entities/Assignment.cs`
 - `backend/Entities/Submission.cs`
 - `backend/Entities/SubmissionStatus.cs`
 - `backend/Configurations/AssignmentConfiguration.cs` (EF configuration)
-

13. Functionality

- Create / edit / delete assignments per classroom
- Upload submissions with strict file validation (type, size, MIME match)
- Replace and cancel submissions under allowed statuses
- Late-submission detection
- Status lifecycle: SUBMITTED → UNDER_REVIEW → REVIEWED → GRADED / RETURNED_FOR_RESUBMISSION → SUBMITTED_AGAIN
- Grade and feedback entry
- Return for resubmission
- Publish evaluation gate (visible to students only after publishing)
- Preview and download submission files with authorization
- Roster-style submission listing for instructors (including non-submitters)
- Audit logging of assignment/submission lifecycle events